UNIVERSITÉ HASSAN II DE CASABLANCA
École Normale Supérieure de l’Enseignement Technique

COMPTE RENDU DE TRAVAUX PRATIQUES

TP 02

Docker Engine
Jenkins & CI/CD

Intégration, livraison et déploiement continus

GIT — BUILD — TEST — PUSH — DEPLOY

Réalisé par

Abdelali Saadali

II BDCC · Ingénierie Informatique, Big Data et Cloud Computing
Module : Ingénierie des Infrastructures Big Data et Cloud
Encadrant : Pr. Kamal EL GUEMMAT

ANNÉE UNIVERSITAIRE 2026 / 2027

15 OCTOBRE 2026

OUVERTURE

Sommaire et synthèse

Ce document restitue la réalisation du TP 2 : Docker Engine, Jenkins, CI/CD de l’ENSET Mohammedia. Les figures sont issues des captures réalisées pendant les manipulations ; les extraits de code reflètent le dépôt GitHub tp4 et les commandes effectivement utilisées.

Sommaire

Rubrique	Pages
1. Objectifs, environnement et architecture	3–4
2. Partie 1 : construction et publication	5–8
3. Partie 2 : déploiement et pipelines	9–14
4. Incidents, récupération et validation finale	15–16
5. Conclusion et ressources	17

Résultat global

CHAÎNE CI/CD FONCTIONNELLE

L’application Welcome BDCC est conteneurisée sous Nginx, construite via Jenkins, publiée sur Docker Hub sous le tag latest et déployée dans Docker Engine. Les pipelines demandés ont été exécutés avec succès.

La partie 1 porte sur le job Freestyle job1tp4 et sa suppression. La partie 2 met en œuvre job2tp4, le Pipeline déclaratif job2tp4v2 et le Pipeline depuis SCM job3tp4. Les modifications successives de index.html permettent de vérifier la prise en compte des nouveaux commits.

Périmètre : mise en œuvre dans un GitHub Codespace, avec accès aux interfaces Jenkins et Nginx par des ports redirigés. Les constructions Jenkins sont lancées manuellement ; la présence d’un webhook de déclenchement automatique n’est pas attestée.

1 · CADRE DU TP

Objectifs et environnement

Objectifs pédagogiques

- Créer une application web minimale et son Dockerfile Nginx.
- Versionner le projet sur GitHub et récupérer son code depuis Jenkins.
- Construire, tester et publier une image Docker sur Docker Hub.
- Déployer un conteneur, puis vérifier les changements après un commit.
- Différencier jobs Freestyle, Pipeline inline et Pipeline défini dans jenkinsfile.

Plateforme de réalisation

Composant	Choix constaté dans le TP
Environnement de travail	GitHub Codespaces / VS Code Web
Gestion du code	GitHub : AbdelaliSaadali/tp4 (branche main)
CI/CD	Jenkins, travaux répartis entre trois jobs conservés
Moteur de conteneurs	Docker Engine, via /var/run/docker.sock
Application	Nginx 1.27.4-alpine + index.html
Registre d’images	Docker Hub : abdelalisaadali/tp4:latest
Accès aux services	Jenkins : 8080 ; Nginx : 8081 / 8082 / 8083

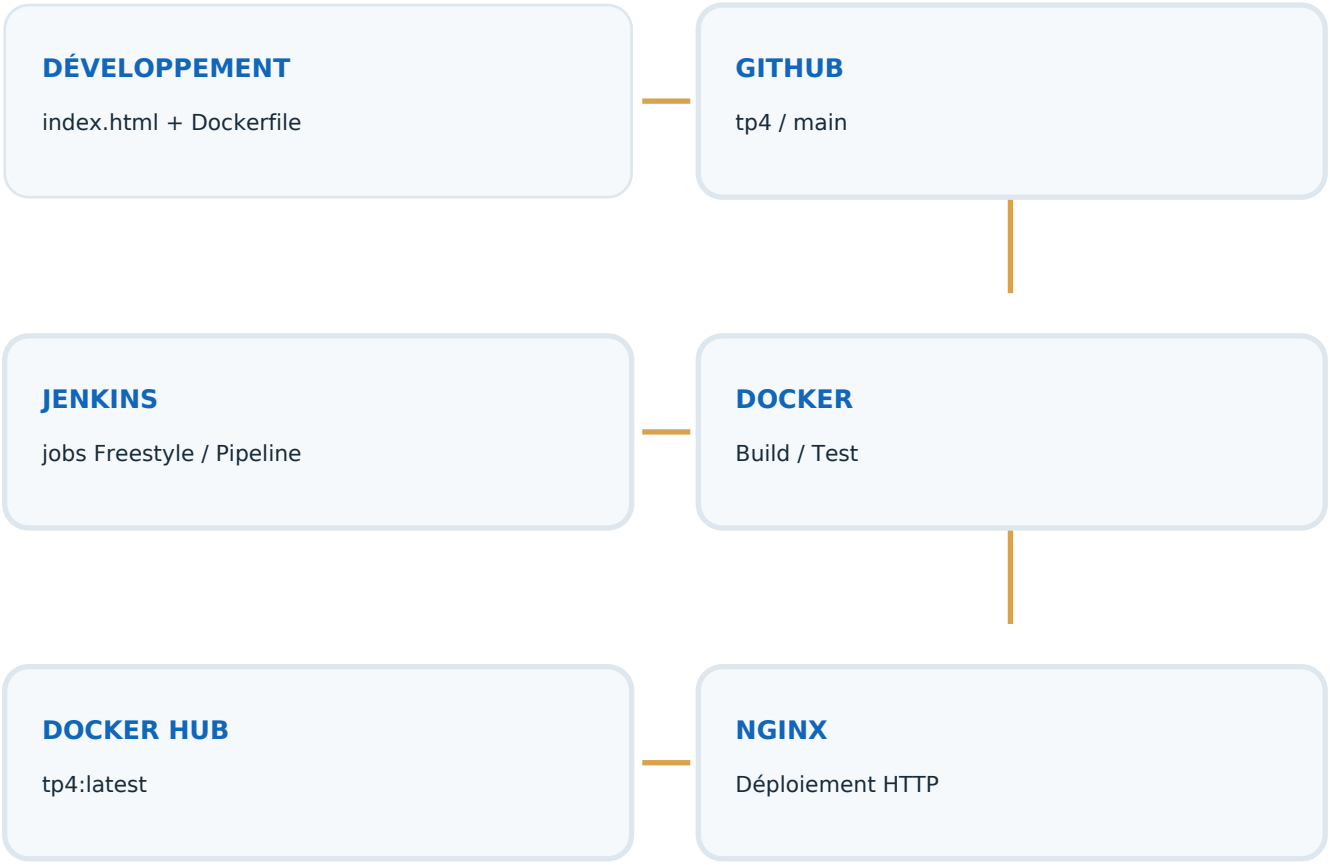
PERSONNALISATION EXIGÉE

Les identifiants des ressources, les journaux Jenkins et la page HTML font apparaître le projet tp4, le compte GitHub et la mention Abdelali Saadali.

1 · ARCHITECTURE

Du commit Git à la page déployée

La chaîne de réalisation sépare le code source (GitHub), l’orchestration (Jenkins), la fabrication de l’image (Docker) et sa distribution (Docker Hub). Le déploiement consomme ensuite l’image publiée.



Logique des étapes

1. Changement du code HTML → 2. récupération Git → 3. construction de l’image → 4. contrôle du fichier embarqué → 5. connexion à Docker Hub et push → 6. démarrage / remplacement du conteneur Nginx.

VÉRIFICATIONS TECHNIQUES

Des tests grep recherchent Welcome BDCC dans le document embarqué dans l’image puis dans le conteneur. La sortie Jenkins confirme ensuite la publication et le déploiement.

2 · PARTIE 1

Application et conteneur Nginx

Structure du dépôt GitHub tp4

Le dépôt contient notamment index.html, Dockerfile, README.md et, après la seconde partie, le fichier jenkinsfile. La page HTML affiche l’accueil attendu et l’identité de l’étudiant.

Dockerfile utilisé

```
FROM nginx:1.27.4-alpine
LABEL org.opencontainers.image.authors="Abdelali Saadali"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

L’image de base fournit Nginx. L’instruction COPY place la page dans le répertoire public du serveur ; EXPOSE 80 documente le port HTTP interne du conteneur.

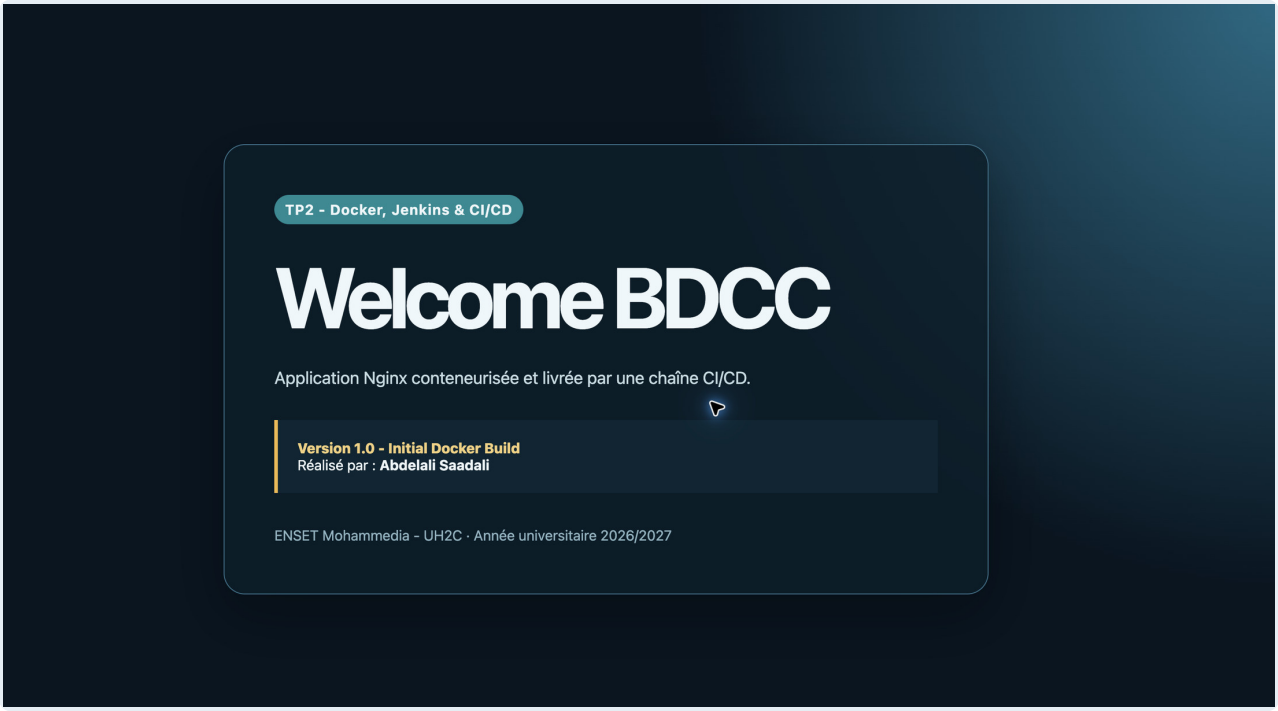


FIG. 01 Première interface Nginx personnalisée, affichant « Welcome BDCC » et « Abdelali Saadali » (version initiale).

2 · PARTIE 1

Jenkins et accès au registre

Le job job1tp4 est créé en mode Freestyle avec la récupération du dépôt GitHub. Les étapes shell pilotent la construction Docker et le push vers le registre. Le branchement d’identifiants évite de placer le token en clair dans le script.

Association des identifiants au job

Élément	Configuration
Type de binding	Username and password (separated)
ID Jenkins	dockerhub-tp2
Variable utilisateur	DOCKER_USER
Variable secrète	DOCKER_TOKEN

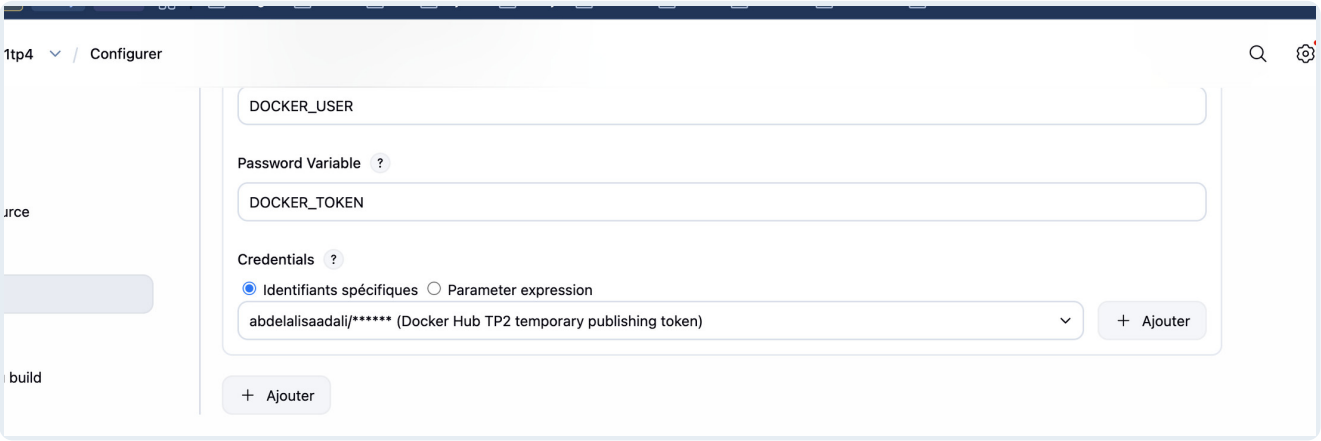


FIG. 02 Configuration « Environnement » du Freestyle : variables et identifiant Jenkins sélectionné (valeur masquée).

L’illustration montre une étape de configuration. La version opérationnelle du script et la réussite de publication sont attestées par les journaux postérieurs. Le secret n’est pas reproduit dans le rapport.

2 · PARTIE 1

job1tp4 : build et push Docker Hub

Après correction de l’identifiant Docker Hub, le build job1tp4 #6 a produit et publié l’image abdelalisaadali/tp4:latest. Le journal contient la signature du push et se termine par Finished: SUCCESS.

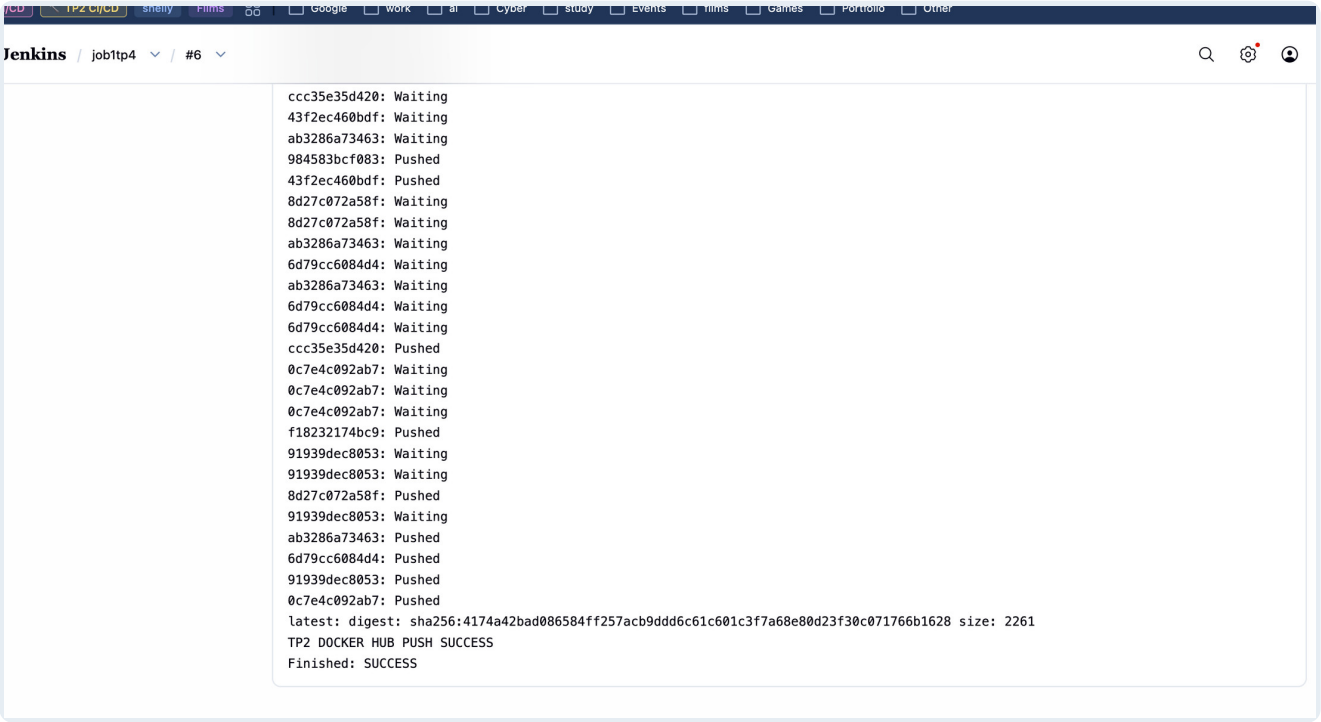


FIG. 03 Journal authentique du build #6 : tag latest, digest SHA-256 et confirmation de succès.

Séquence de publication

```
docker build -t tp4:latest .
printf '%s' "$DOCKER_TOKEN" | docker login \
  -u "$DOCKER_USER" --password-stdin
docker tag tp4:latest "$DOCKER_USER/tp4:latest"
docker push "$DOCKER_USER/tp4:latest"
```

L’authentification utilise Jenkins Credentials Binding. Les variables DOCKER_USER et DOCKER_TOKEN ne contiennent pas de valeurs codées en dur dans le script.

2 · PARTIE 1

Registre, commit et suppression du job

Publication disponible sur Docker Hub

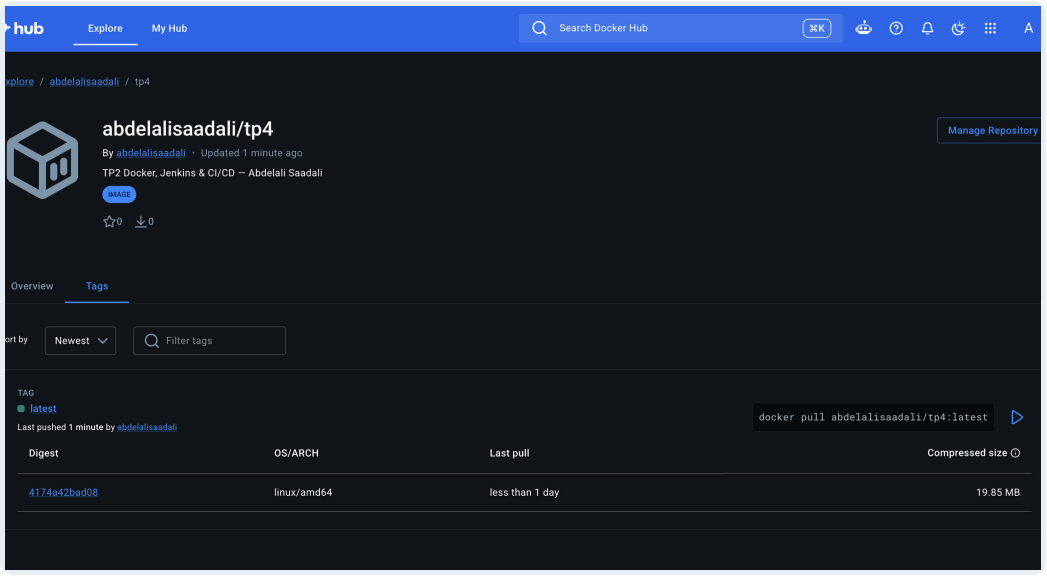


FIG. 04 Référentiel Docker Hub « abdelalisaadali/tp4 » avec le tag latest disponible.

Une première mise à jour du fichier index.html introduit la mention Version 2.1 — CI/CD Update Verified. Le build job1tp4 #7 s'exécute après cette modification ; l'interface Jenkins le signale en état vert.

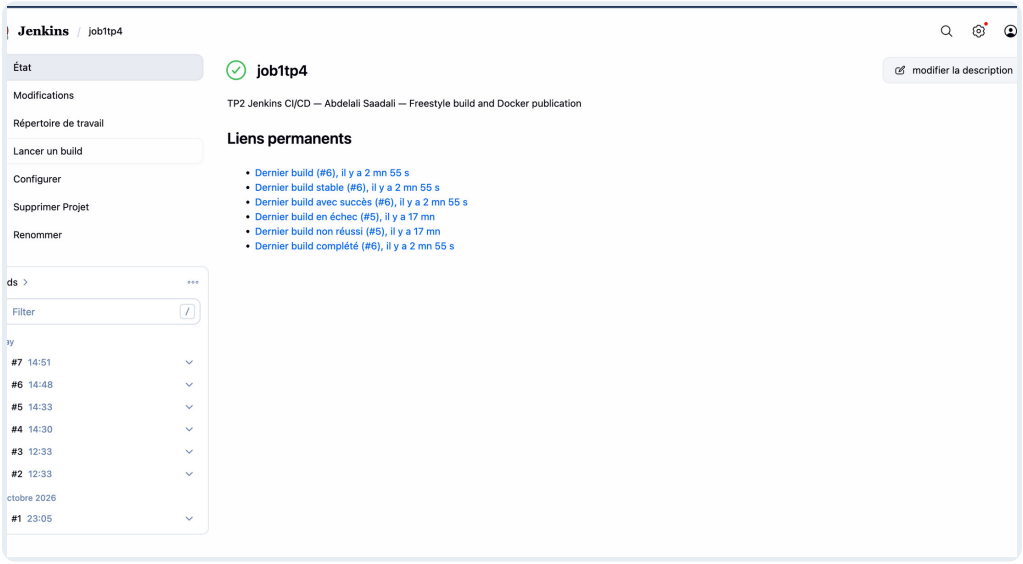


FIG. 05 Historique de job1tp4 présentant le build #7 en succès après la modification du HTML.

FIN DE LA PARTIE 1

Le job Freestyle job1tp4 a ensuite été supprimé, conformément à la consigne. La dernière vue d'ensemble du serveur Jenkins ne répertorie plus ce job.

3 · PARTIE 2

job2tp4 : Freestyle avec déploiement

Le job job2tp4 prolonge les actions précédentes par un démarrage effectif de l’image publiée dans Docker Engine. L’application est exposée sur le port hôte 8083.

Chaîne exécutée

Action	Commande ou contrôle représentatif
Clonage	SCM Git : branche main
Construction	DOCKER_BUILDKIT=0 docker build --pull --no-cache -t tp4:latest .
Test	Recherche de « Welcome BDCC » dans index.html embarqué
Publication	docker push abdelalisaadali/tp4:latest
Déploiement	docker run -d --name tp4-deploy -p 8083:80 ...
Validation	docker inspect + docker exec + code retour Jenkins SUCCESS

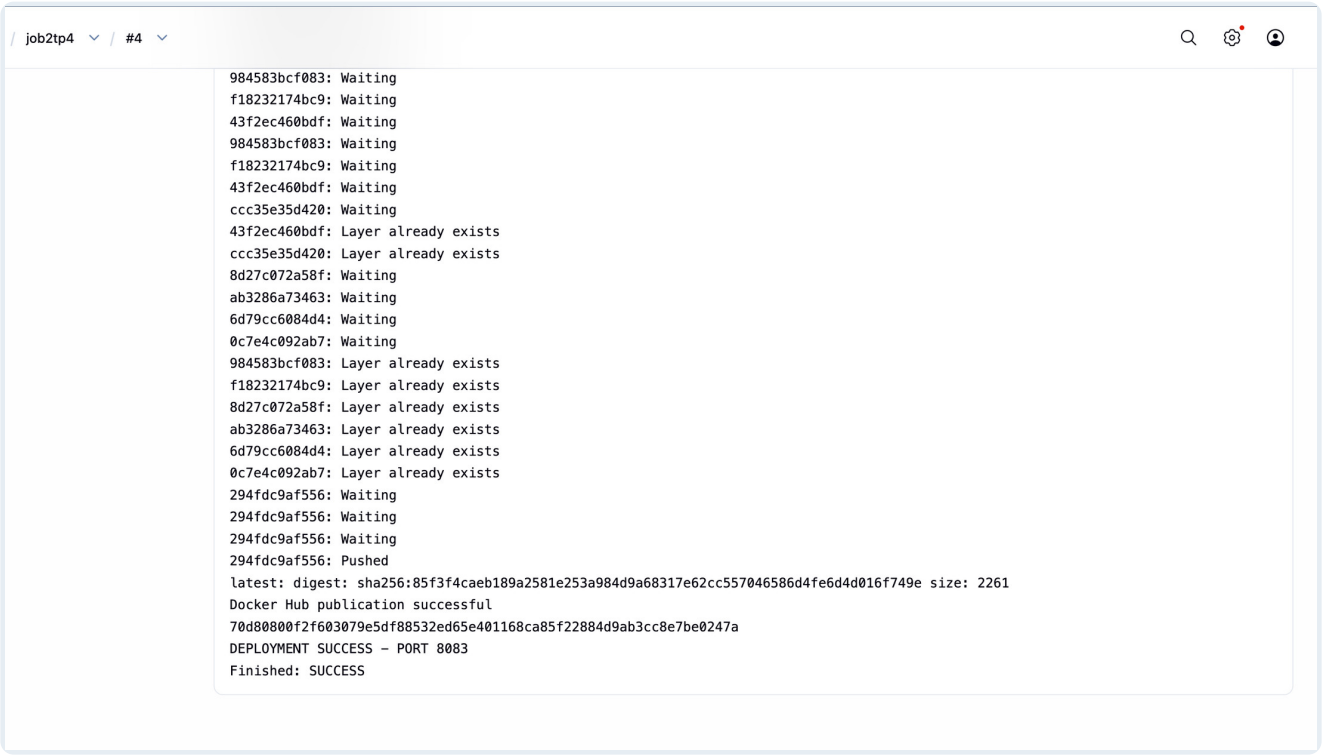


FIG. 06 Extrait du journal job2tp4 #4 : digest latest, publication Docker Hub et « DEPLOYMENT SUCCESS - PORT 8083 ».

La désactivation ponctuelle de BuildKit répond à un problème de cache Docker rencontré après une panne de stockage, traité plus loin.

3 · PARTIE 2

Mise à jour HTML et redéploiement

Validation d’un nouveau contenu

Une seconde modification du dépôt remplace la mention de version par Version 2.2 — Automatic Redeployment Verified. Le job Freestyle est relancé ; la capture finale montre le nouveau contenu sur le service déployé 8083.

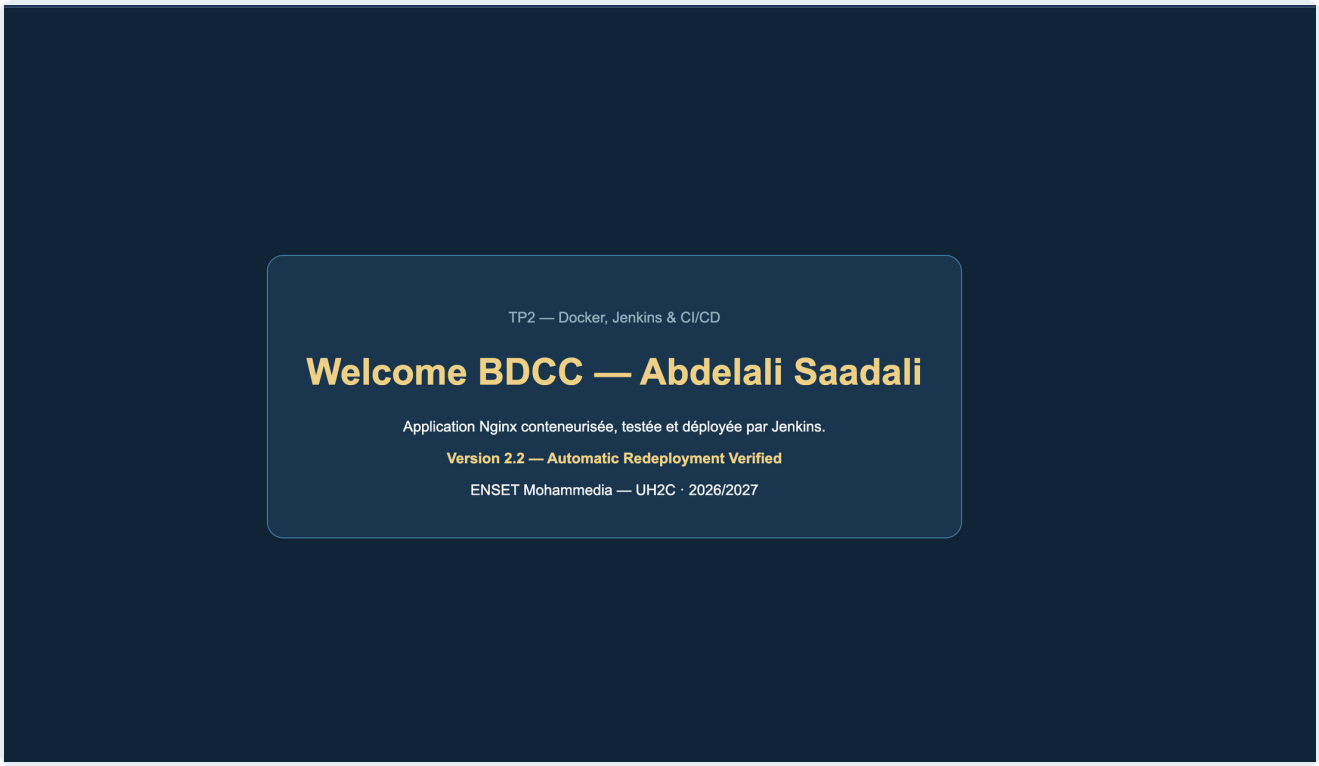


FIG. 07 Application Nginx redéployée depuis job2tp4 : Version 2.2 visible, identité affichée.

CRITÈRE DE VALIDATION

La comparaison du contenu versionné et de la page servie établit que le déploiement reflète la modification de index.html. Le tableau de bord final présente job2tp4 #5 comme dernier build réussi.

Logique de redéploiement

```
docker rm -f tp4-deploy >/dev/null 2>&1 || true
docker run -d --name tp4-deploy -p 8083:80 \
  "$DOCKER_USER/tp4:latest"
docker inspect -f '{{.State.Running}}' tp4-deploy
```

3 · PARTIE 2

job2tp4v2 : Pipeline inline

La consigne demande un Pipeline défini directement dans le champ Pipeline script et limité à trois stages intitulés exactement : Cloning Git, Building image et Publish Image. La version finale du job a été exécutée avec succès.

Architecture du Pipeline

Stage	Tâche
Cloning Git	Extraction du code GitHub (main)
Building image	Construction locale de tp4:latest
Publish Image	withCredentials, connexion sécurisée, tag et docker push

Extrait de la logique opérationnelle

```
pipeline {
  agent any
  stages {
    stage('Cloning Git') { steps { git branch: 'main',
      url: 'https://github.com/AbdelaliSaadali/tp4.git' } }
    stage('Building image') { steps { sh 'docker build -t tp4:latest .' } }
    stage('Publish Image') { steps { /* withCredentials + docker push */ } }
  }
}
```

Le journal communiqué pour l’exécution conforme indique Login Succeeded, la publication du tag latest, puis JOB2TP4V2 - PUBLISH SUCCESS et Finished: SUCCESS.

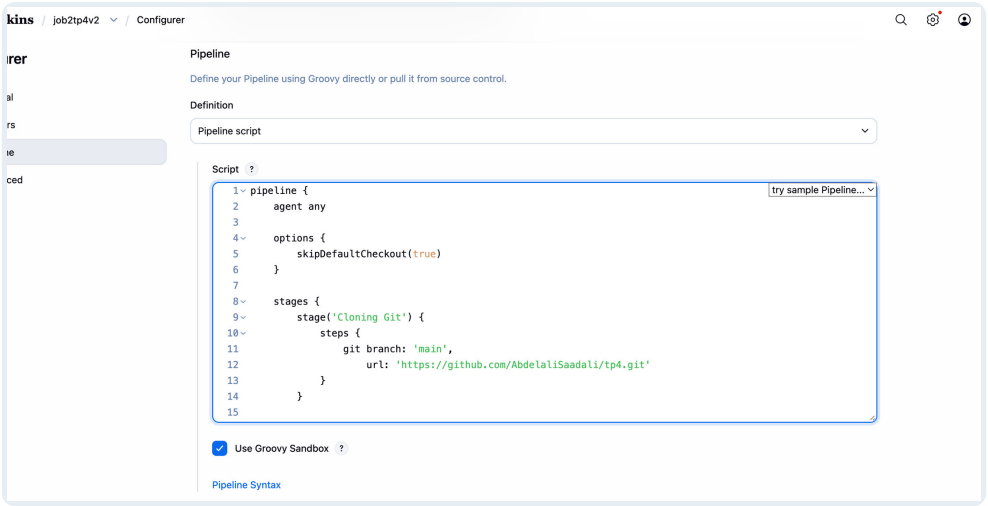


FIG. 08 Interface Jenkins « Pipeline script » : configuration inline du job2tp4v2 (extrait du script).

Le ZIP contient également une ancienne capture d’un pipeline « Checkout / Build / Test ». Elle correspond à une étape intermédiaire et n’est pas présentée comme la validation finale. La réussite du pipeline à trois stages est établie par la sortie console fournie.

3 · PARTIE 2

job3tp4 : quatre stages depuis GitHub

Le job job3tp4 utilise Pipeline script from SCM, pointant vers le fichier jenkinsfile à la racine du dépôt. La première version exécute les quatre étapes exigées par l'énoncé.

Quatre stages demandés

Ordre	Nom du stage	Résultat vérifié
1	Cloning Git	Récupération de main
2	Building image	Docker build exécuté
3	Test image	Recherche de « Welcome BDCC » dans l'image
4	Publish Image	Push latest sur Docker Hub

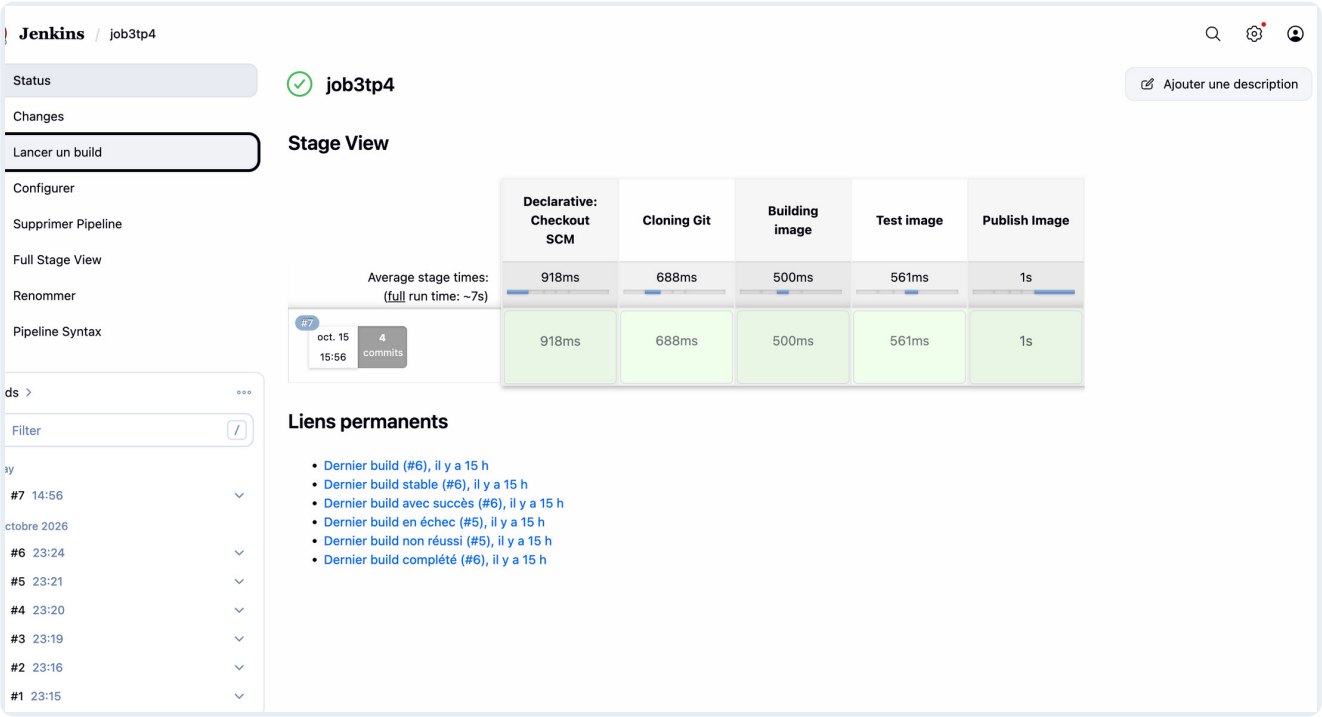


FIG. 09 Stage View de job3tp4 #7 : quatre étapes du TP en vert ; la colonne « Declarative: Checkout SCM » est un checkout technique ajouté par Jenkins.

PREUVE DE PUBLICATION

Le journal de ce build contient « Obtained jenkinsfile from git », « Login Succeeded », un digest SHA-256 pour latest et Finished: SUCCESS.

3 · PARTIE 2

job3tp4 : cinq stages et déploiement

Le jenkinsfile a ensuite été enrichi par le stage deploy image. Son rôle est de remplacer le conteneur tp4-final, de l'exposer en 8082:80 et de contrôler son état ainsi que le HTML servi.



FIG. 10 Stage View de job3tp4 : Cloning Git, Building image, Test image, Publish Image et deploy image, tous réussis.

Bloc de déploiement ajouté

```
docker rm -f tp4-final >/dev/null 2>&1 || true
docker run -d --name tp4-final -p 8082:80 \
  abdelalisaadali/tp4:latest
docker inspect -f '{{.State.Running}}' tp4-final | grep -q true
docker exec tp4-final sh -c \
  'grep -q "Welcome BDCC" /usr/share/nginx/html/index.html'
```

La sortie console associe la ligne « DEPLOYMENT SUCCESS - PORT 8082 » à « Finished: SUCCESS ». La colonne « Declarative: Checkout SCM » n'est pas un sixième stage métier.

3 · PARTIE 2

Contrôle du conteneur déployé

Le déploiement de job3tp4 produit un second site Nginx disponible via le port redirigé 8082. Cette étape complète la chaîne de livraison en ajoutant une vérification applicative post-déploiement.

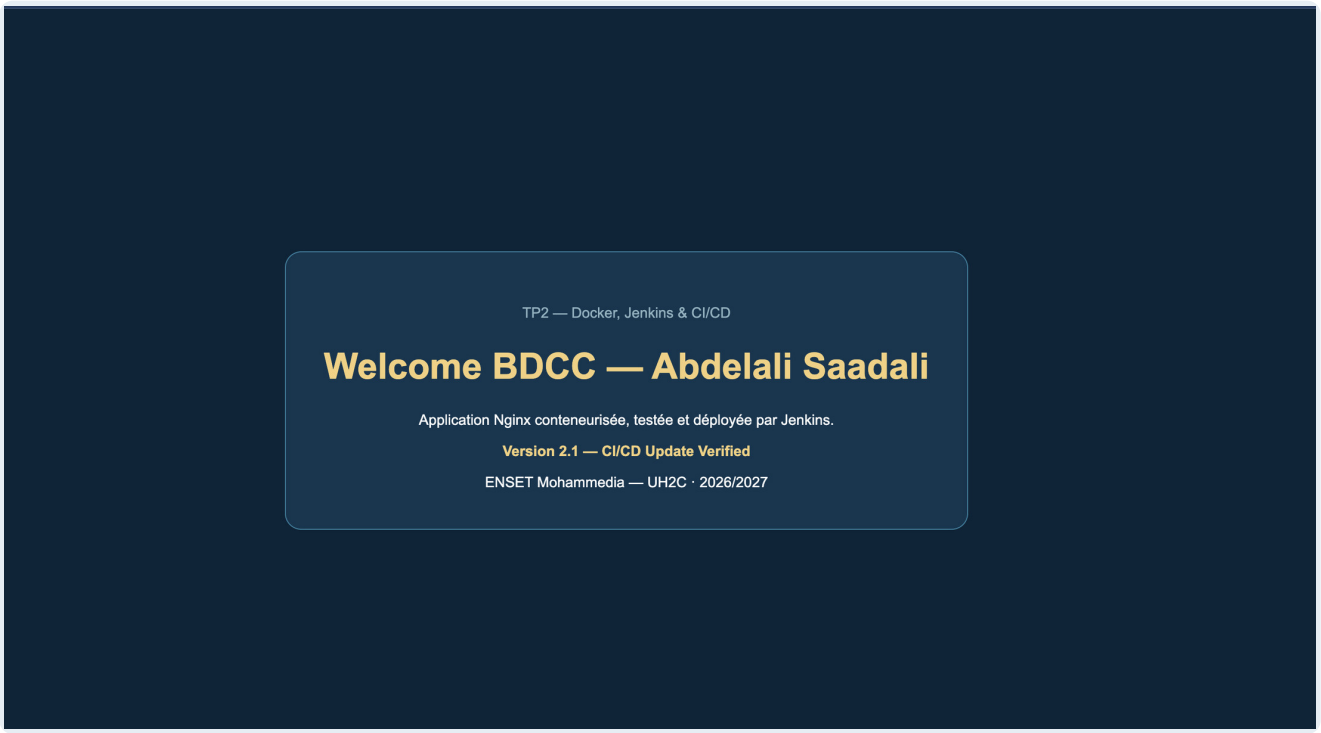


FIG. 11 Page web issue du Pipeline job3tp4, affichée sur le service déployé en port 8082 ; version 2.1 visible au moment de la capture.

Comparaison des deux stratégies

Mode	Propriété clé	Cible
Freestyle job2tp4	Étapes de construction dans l'interface Jenkins	8083
Pipeline job3tp4	Script CI/CD versionné dans jenkinsfile sur GitHub	8082

Les deux cibles sont distinctes et ne doivent pas être confondues : port 8082 pour le Pipeline GitHub ; port 8083 pour le Freestyle de déploiement.

4 · EXPLOITATION

Incident Docker et récupération Jenkins

Une interruption de l’environnement Codespaces a arrêté plusieurs conteneurs. Jenkins n’était plus accessible (HTTP 502 côté navigateur). L’analyse a montré un problème de couche écrivable et de cache Docker, sans remettre en cause les scripts applicatifs.

Diagnostic et résolution

Symptôme	Traitement réalisé
jenkins-tp4 : Exited (255), RWLayer unexpectedly nil	Inspection des volumes, puis vérification de la persistance de jenkins_home
Image locale tp4-jenkins:local indisponible	Sauvegarde du volume avec Alpine ; reconstruction d’une image Jenkins
docker: not found dans le job restauré	Installation du paquet docker-cli dans le conteneur
BuildKit : parent snapshot does not exist	Reconstruction avec DOCKER_BUILDKIT=0, --pull et --no-cache

Traçabilité des opérations

```
docker volume inspect jenkins_home
docker run --rm -v jenkins_home:/source:ro ... alpine:3.20 \
  sh -c 'tar -czf /backup/jenkins_home_backup.tar.gz -C /source .'
docker exec -u root jenkins-tp4 apt-get install -y docker-cli
DOCKER_BUILDKIT=0 docker build --pull --no-cache -t tp4:latest .
```

RÉSULTAT DE RÉCUPÉRATION

La sauvegarde a affiché JENKINS BACKUP VERIFIED. Les projets Jenkins ont été retrouvés et le job2tp4 a finalement terminé avec Finished: SUCCESS.

Durabilité : les paquets installés manuellement dans un conteneur doivent être intégrés au Dockerfile de l’image Jenkins si une nouvelle recréation est prévue. Les sauvegardes du volume Jenkins restent privées.

4 · VALIDATION FINALE

Bilan des exigences et tableau de bord

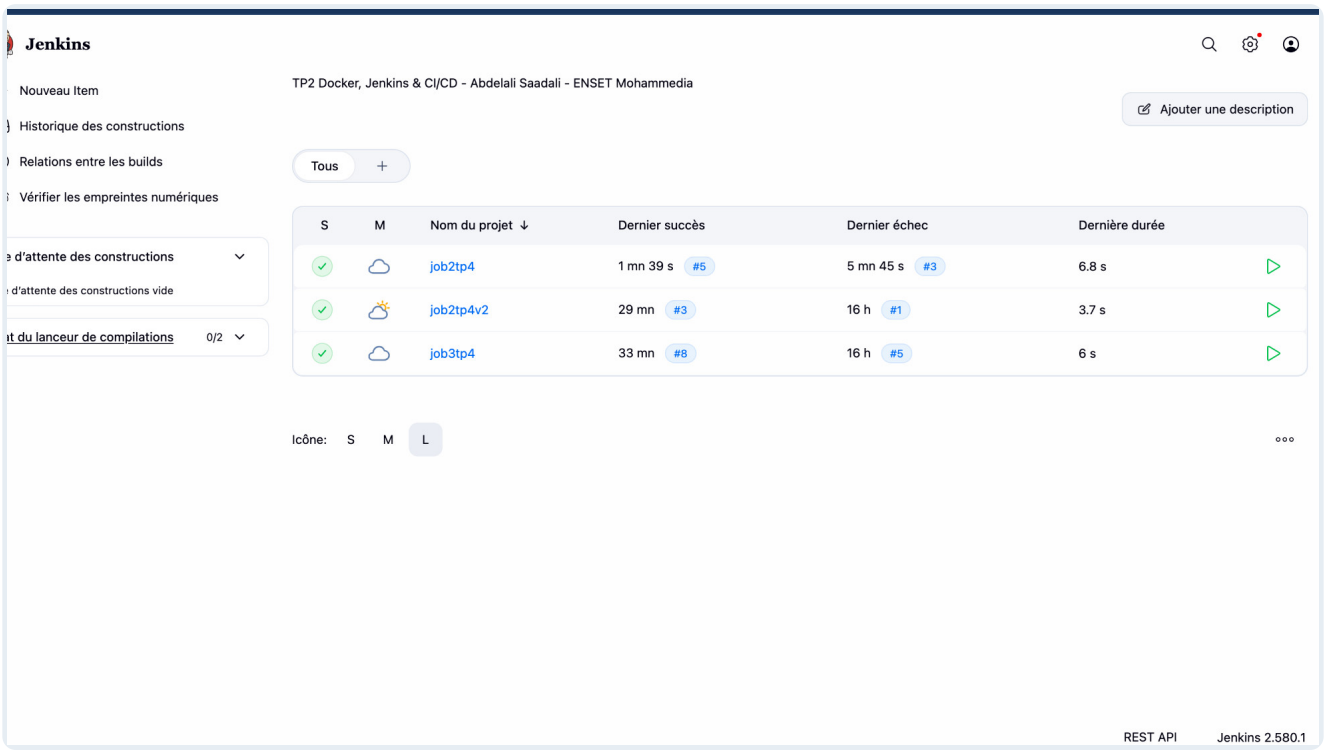


FIG. 12 Tableau de bord Jenkins final : job2tp4, job2tp4v2 et job3tp4 sont verts ; job1tp4 n'apparaît plus.

État de conformité

Exigence	Résultat	Justification
Application GitHub + Dockerfile Nginx	Validé	Dépôt GitHub et page Welcome BDCC
Freestyle job1tp4 + Docker Hub latest	Validé	Build #6, Docker Hub Tags
Modification HTML puis suppression job1tp4	Validé	Build #7 et dashboard sans job1tp4
Freestyle job2tp4 et redéploiement	Validé	Port 8083 ; Version 2.2 ; job2tp4 #5
job2tp4v2 : Pipeline inline, trois stages	Validé	Sortie du build final : Publish SUCCESS
job3tp4 : quatre puis cinq stages	Validé	Stage Views et jenkinsfile sur GitHub

Réserve méthodologique : le ZIP ne comprend pas une capture de la Stage View finale à trois stages de job2tp4v2, mais la sortie console correspondante a été communiquée et vérifiée.

5 · CONCLUSION

Synthèse, limites et ressources

Conclusion

Ce TP a permis de construire une chaîne CI/CD complète pour une application Nginx : gestion du code source avec GitHub, intégration via Jenkins, création et validation d'images Docker, publication sur Docker Hub et déploiement dans Docker Engine. L'objectif demandé en deux parties est atteint, avec un passage progressif d'un Freestyle simple à un Pipeline versionné et extensible.

Les captures finales confirment l'existence du tag latest, le fonctionnement des quatre puis cinq stages du Pipeline depuis SCM, la disponibilité de l'application sur les ports 8082 et 8083, ainsi que la suppression du job initial. La panne intermédiaire a également illustré l'importance des volumes persistants, des sauvegardes et du diagnostic du moteur Docker.

Points d'attention

- Les exécutions ont été lancées depuis Jenkins après les commits ; les déclencheurs automatiques GitHub ne font pas partie des preuves présentées.
- Le token Docker Hub ne doit pas apparaître dans les captures ; la révocation du token temporaire exposé précédemment reste recommandée.
- Conserver une image Jenkins reproductible avec Git et Docker CLI, et sauvegarder régulièrement le volume jenkins_home.

Ressources et livrables

Ressource	Référence
Énoncé du TP	Pr. Kamal EL GUEMMAT, ENSET Mohammedia, 2026/2027
Dépôt GitHub	github.com/AbdelaliSaadali/tp4
Image Docker	hub.docker.com/r/abdelalisaadali/tp4
Documentation Jenkins	jenkins.io
Pièces justificatives	Captures écran du 15/10/2026, journaux de construction et Stage Views

BILAN

Le projet démontre la maîtrise des opérations essentielles de l'intégration, de la livraison et du déploiement continus sur une infrastructure conteneurisée.